

SAKTHIVEL BALAKUMAR - 63957

MEKA VENKATA SRAVANTH BABU - 63919

SAMSUDEEN SABEER RAHMAN – 63920

MICROSYSTEM ELECTRONICS – I.E. 2409

LAB 5 – PART 1

During this project we will:

- Perform a contrast enhancement function
- Measure the time for different parametrizations
 - Sections of C program localization
 - Processor modification
 - Program optimization
- Use a hard IP
- Optimize the hard IP

1]Part 1 – HANDS ON THE SYSTEM – SOFTWARE APPROACH

We have replaced the contents of hello world program by the contents of the file “CopyCam.c” and copied the file “APP_CamParam.h” in the corresponding directory.

After running the program, the output was:



The image from the left is copied by the software to the right.

COPYCAM.C

```
//-----  
// CopyCam.c  
//  
//-----
```

```
#include <stdio.h>  
#include "APP_CamParam.h"  
#include "system.h"
```

```

#include "alt_types.h"
#include "altera_avalon_pio_regs.h"
#include "altera_avalon_performance_counter.h"
S_IP_INIT Sparam;
//define USE_FLOAT
// Choisir l'exposition en fonction de la luminosité
// valeur entre 0x0800 et 0x6000 par pas de 0x0100 ou 0x1000
#define EXPO_TIME 0x600
int main()
//-----
//-----
{
    int nerr; uint16 Version;

    printf("\n ----- Hello from Nios II/e. Test LCD + Camera VT ----- \n");

    Sparam.ImageNbrPx = CAM_NBPIX; Sparam.ImageNbrLn = CAM_NBLIN; // taille de l'image du
    capteur
    Sparam.AfiNbrPx = FEN_NBPIX; Sparam.AfiNbrLn = FEN_NBLIN; // taille de l'affichage
    Sparam.AfiZoom = 1; Sparam.CamFocus = 0;
    Sparam.Exptime = EXPO_TIME; Sparam.Expgain = 0x0200;
    Sparam.Color = COLOR_BGR_FOND;

    DE10Sys_Initialize(&Sparam, &Version);

    printf("\n --- LCD Hard IP version read = 0x%04X \n", Version & 0x0FFFF);

    nerr = DE10Sys_InitInterrupt(); // init des interruptions boutons, switches
    nerr += DE10Sys_InitTactile(); // init des interruptions écran tactile
    //
    if (nerr == 0) {
        DE10Sys_AffiEnable();
        Affi_Pattern();
        DE10CAM_RunCapture(); // activer la caméra
        App_UserLoop(&Sparam);
    }
    else
        printf("\n ++++++ erreur d'initialisation des interruptions ++++++ \n\n");

    printf("\n ----- Fin de l'application ----- \n");

    return 0;
}

volatile int *MyIP_Addres;
void My_Copy(void);
void App_UserLoop(S_IP_INIT* param)
//-----
//-----
{
    alt_u64 counter; int time; //short result;

```

```

float im;
MyIP_Addres = (int*)MYIP_BUSMAP;

while (1) {
    PERF_RESET(PERFCOUNTER_BASE);
    PERF_START_MEASURING(PERFCOUNTER_BASE);
    PERF_BEGIN(PERFCOUNTER_BASE, 1);
    My_Copy();
    PERF_END(PERFCOUNTER_BASE, 1);
    counter = perf_get_section_time((void*)PERFCOUNTER_BASE, 1);
    time = counter / 1000;
    printf("Number of cycles = %lld - ", counter);
    im = (float) 50 / ((float) time / 1000);
    printf("%f images / sec\n", im);
    Test_Touche(NULL);
}
}

```

```

void My_Copy(void)
//-----
//-----
{  uint32 pix, npix;
   uint32 *adpixin, *adpixon;
   //int lin,col;
   int i,j;
   //int g,r,b,y;

   for (i=0;i<FEN_NBLIN;i++) {
       adpixin = (uint32 *)DE10Affi_GetAdresse(0, i);
       adpixon = adpixin + DST_DEBPIX;    // 400 pixels further
       for(j=0;j<FEN_NBPIX;j++) {
           pix = *adpixin++;
           npix = pix;
           *adpixon++ = npix;
       }
   }
}

```

In this “CopyCam.c” program the parameter EXPO_TIME is set to 1000,so that we are modifying it depending on the luminosity at our place to get a correct contrasted image.

Modified line in that program:

```
#define EXPO_TIME 0x600 → #define EXPO_TIME 0x1000
```

“App_UserLoop” function:

The **App_UserLoop()** function continuously measures the performance of **My_Copy()** using a hardware timer.

My_Copy() function reads each pixel line-by-line and it writes 400 pixels forward to another location in memory. It is actually used to test the image copying speed and to display the performance in an embedded system.

Q1) How many lines and how many columns are coming from the camera?

Ans:

```
// Dimensions du capteur de la camera
#define CAM_NBPIX      832
#define CAM_NBLIN      484
```

Interpretation: The camera captures an image with 484 lines (rows) and 832 columns (pixels per line).

Q2) Use the performance counter to display the calculation time of the function "MyCopy()". (And report it).

Ans:

The **App_UserLoop()** function uses the performance counter to measure My_Copy() by:

```
PERF_BEGIN(PERFCOUNTER_BASE, 1);
```

```
My_Copy();
```

```
PERF_END(PERFCOUNTER_BASE, 1);
```

```
counter = perf_get_section_time((void*)PERFCOUNTER_BASE, 1);
```

Then it displays the result using:

```
printf("Number of cycles = %lld - %f images / sec\n", counter, im);
```

For EXPO_TIME = 0x600, We are taking the last 10 values of the output

Number of cycles = 27309364 - 1.830898 images / sec

Number of cycles = 27433028 - 1.822622 images / sec

Number of cycles = 27390996 - 1.825484 images / sec

Number of cycles = 27174743 - 1.839994 images / sec

Number of cycles = 27189607 - 1.838979 images / sec

Number of cycles = 27343994 - 1.828622 images / sec

Number of cycles = 27446500 - 1.821759 images / sec

Number of cycles = 27366522 - 1.827085 images / sec

Number of cycles = 27147793 - 1.841824 images / sec

Number of cycles = 27213130 - 1.837357 images / sec

SUM = 273915677

Average :

= 273915677 / 10 = 27,391,568 cycles

The function My_Copy() takes an average of 27, 391, 568 clock cycles. So there are 1.83 images per second.

For EXPO_TIME = 0x1000, We are taking the last 10 values of the output

Number of cycles = 25877673 - 1.932218 images / sec

Number of cycles = 25979827 - 1.924631 images / sec

Number of cycles = 25883848 - 1.931770 images / sec

Number of cycles = 25810627 - 1.937234 images / sec

Number of cycles = 25909825 - 1.929831 images / sec

Number of cycles = 26049180 - 1.919459 images / sec

Number of cycles = 25967956 - 1.925521 images / sec

Number of cycles = 25876429 - 1.932292 images / sec

Number of cycles = 25890118 - 1.931248 images / sec

Number of cycles = 26000156 - 1.923077 images / sec

SUM = 2,592,456,239

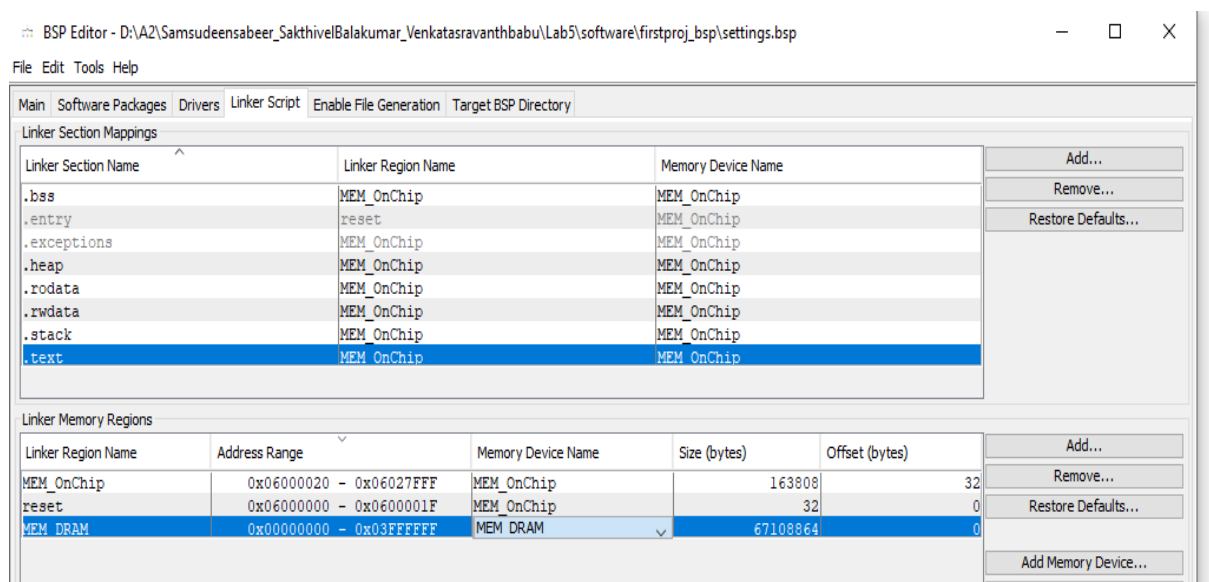
Average :

= 2,592,456,239 / 10 = 259,245,623.9 cycles

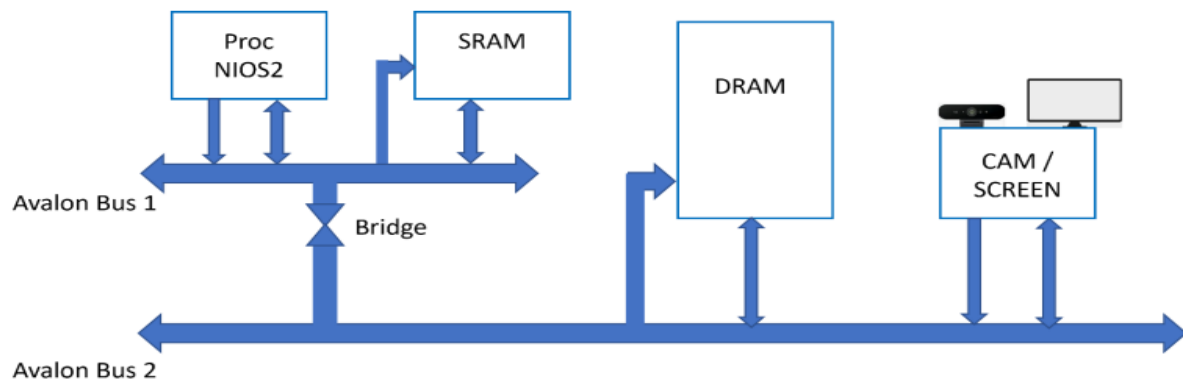
The function My_Copy() takes an average of 259,245,623.9 clock cycles. So there are 1.93 images per second.

Interpretation: The PERF_COUNTER is used to measure how many clock cycles the function MyCopy() takes to execute, helping to evaluate performance.

Setting all section to Mem_OnChip:



BSP is Regenerated.



Q3) What is now the function time? (Explain why).

Ans: The function time is high because the grayscale conversion in `My_Copy_Grayscale()` uses floating-point operations for every pixel. Hence the Y(luminance) component is calculated using:

$$Y = 0.21 * R + 0.72 * G + 0.07 * B$$

The floating-point calculations are slow on Nios II/e, which increases the total processing time.

Q4) What is the formula to get Y from RGB parameters?

Ans: The formula to get Y (luminance) from RGB is:

$$Y = 0.299 * R + 0.587 * G + 0.114 * B$$

Explanation: This formula accurately reflects how the human eye perceives brightness, with green having the highest.

Red: 29.9%, **Green:** 58.7%, **Blue:** 11.4%

$Y = 0.21 * R + 0.72 * G + 0.07 * B \rightarrow$ this formula is less accurate.

Q5) Write a program to display the Black and White image at the right. (You'll use floating point numbers to establish the conversion).

Ans: Program:

```

//-----
//      CopyCam.c
//
//-----
#include <stdio.h>
#include "APP_CamParam.h"
#include "system.h"
#include "alt_types.h"
#include "altera_avalon_pio_regs.h"
#include "altera_avalon_performance_counter.h"
S_IP_INIT    Sparam;
//#define    USE_FLOAT
// Choisir l'exposition en fonction de la luminosité
// valeur entre 0x0800 et 0x6000 par pas de 0x0100 ou 0x1000
#define      EXPO_TIME    0x600

int          main()

```

```

//-----
//-----
{
    int nerr;    uint16 Version;

    printf("\n ----- Hello from Nios II/e. Test LCD + Camera VT
----- \n");

    Sparam.ImgNbrPx = CAM_NBPIX; Sparam.ImgNbrLn = CAM_NBLIN; // taille
de l'image du capteur
    Sparam.AfiNbrPx = FEN_NBPIX; Sparam.AfiNbrLn = FEN_NBLIN; // taille
de l'affichage
    Sparam.AfiZoom = 1;                Sparam.CamFocus = 0;
    Sparam.Exptime = EXPO_TIME; Sparam.Expogain = 0x0200;
    Sparam.Color = COLOR_BGR_FOND;

    DE10Sys_Initialize(&Sparam, &Version);

    printf("\n          --- LCD Hard IP version read = 0x%04X \n",
Version & 0x0FFFF);

    nerr = DE10Sys_InitInterupt();           // init des interruptions
boutons, switches
    nerr += DE10Sys_InitTactile( );           // init des interruptions
ecran tactile
                                           //

    if (nerr == 0) {
        DE10Sys_AffiEnable();
        Affi_Pattern();
        DE10CAM_RunCapture();                // activer la caméra
        App_UserLoop(&Sparam);
    }
    else
        printf("\n ++++++ erreur d'initialisation des interruptions
++++++ \n\n");

    printf("\n ----- Fin de l'application ----- \n");

    return 0;
}
volatile int *MyIP_Addres;
void My_Copy_Grayscale(void);
void App_UserLoop(S_IP_INIT* param)
//-----
//-----
{
    alt_u64 counter; int n, time;    short result;
    float im;
    MyIP_Addres = (int*)MYIP_BUSMAP;

    while (1) {
        PERF_RESET(PERFCOUNTER_BASE);
        PERF_START_MEASURING(PERFCOUNTER_BASE);
        PERF_BEGIN(PERFCOUNTER_BASE, 1);
        My_Copy_Grayscale();
        PERF_END(PERFCOUNTER_BASE, 1);
        counter = perf_get_section_time((void*)PERFCOUNTER_BASE, 1);
        time = counter / 1000;
        printf("Number of cycles = %lld - ", counter);
        im = (float) 50 / ((float) time / 1000);
    }
}

```

```

        printf("%f images / sec\n", im);
        Test_Touche(NULL);
    }
}

// Convert image to grayscale and display on the right
void My_Copy_Grayscale(void)
{
    uint32 pix, gray_pix;
    int B, G, R;
    float Y;
    uint32 *adpixin, *adpixout;
    int i, j;
    int gray;

    for (i = 0; i < FEN_NBLIN; i++) {
        adpixin = (uint32 *)DE10Affi_GetAdresse(0, i);
        adpixout = adpixin + DST_DEBPIX; // 400 pixels to the right

        for (j = 0; j < FEN_NBPIX; j++) {
            pix = *adpixin++;

            // Extract RGB
            B = (pix & 0x000000FF);
            G = (pix & 0x0000FF00) >> 8;
            R = (pix & 0x00FF0000) >> 16;

            // Convert to grayscale using floating point
            Y = 0.21 * R + 0.72 * G + 0.07 * B;
            gray = (int)Y;
            gray &= 0xff;

            // Pack grayscale into RGB format
            gray_pix = (gray << 16) | (gray << 8) | gray;

            *adpixout++ = gray_pix;
        }
    }
}

```

In order to enhance the contrast, you will:

- Store the y minimum value and the y maximum value
- Establish a formula to compute a new pixel value function of those two parameters

```

void My_Copy_Grayscale(void)
{
    uint32 pix, gray_pix;
    int B, G, R;
    float Y, Ymin = 255.0, Ymax = 0.0, Ynew;
    uint32 *adpixin, *adpixout;
    int i, j;
    float Ybuffer[FEN_NBLIN][FEN_NBPIX]; // To store Y values
    int gray;

    // First pass: Calculate Y and find Ymin, Ymax
    for (i = 0; i < FEN_NBLIN; i++) {
        adpixin = (uint32 *)DE10Affi_GetAdresse(0, i);
        for (j = 0; j < FEN_NBPIX; j++) {
            pix = *adpixin++;

```



```

    B = (pix & 0x000000FF);
    G = (pix & 0x0000FF00) >> 8;
    R = (pix & 0x00FF0000) >> 16;

    Y = 0.299 * R + 0.587 * G + 0.114 * B;
    Ybuffer[i][j] = Y;

    if (Y < Ymin) Ymin = Y;
    if (Y > Ymax) Ymax = Y;
}
}

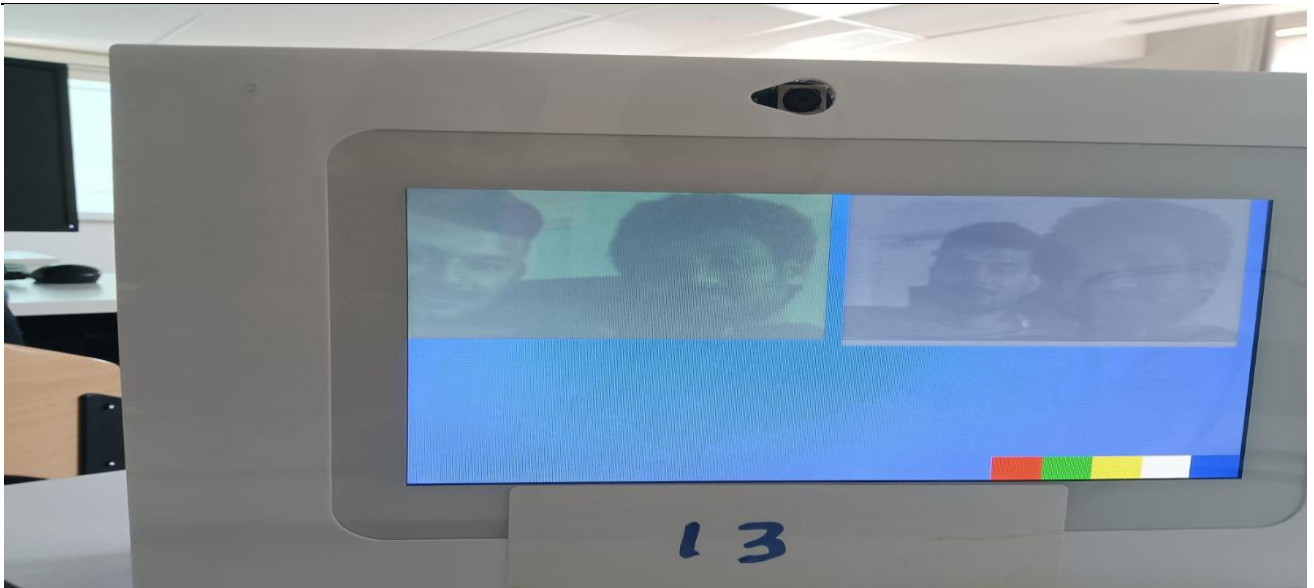
// Second pass: Normalize Y and display enhanced grayscale image
for (i = 0; i < FEN_NBLIN; i++) {
    adpixin = (uint32 *)DE10Affi_GetAdresse(0, i);
    adpixout = adpixin + DST_DEBPIX;

    for (j = 0; j < FEN_NBPIX; j++) {
        Y = Ybuffer[i][j];
        Ynew = (Y - Ymin) / (Ymax - Ymin) * 255.0;
        gray = (int)Ynew;
        gray &= 0xFF;

        gray_pix = (gray << 16) | (gray << 8) | gray;
        *adpixout++ = gray_pix;
    }
}
}

```

Output:



Interpretation: A grayscale image is generated using the floating-point math, then its contrast is improved by normalizing the pixel values based on the minimum and maximum Y (luminance) values.

Q6) What formula will you use?

In order to implement it you will write a program which will:

- Read all pixels in the image to get the Y min and Y max.
- Use those variables to read again / compute / display the new pixel value.

Ans:

To enhance the contrast we will use the formula:

$$Y_{new} = \left(\frac{Y - Y_{min}}{Y_{max} - Y_{min}} \right) * 255$$

Why we are using this formula:

We can scan all the pixels to find Ymin and Ymax. For each pixel we apply this formula to stretch the pixel intensity across the full 0-255 grayscale range. It helps to improve the visual contrast of the final black and white image.

Q7) Program the function to enhance the contrast. Report the calculation time.

Program to enhance the contrast:

```
void My_Copy_Grayscale_Contrast_Int(void)
{
    uint32 pix, gray_pix;
    int R, G, B, Y, Ynew;
    int Ymin = 255, Ymax = 0;
    int i, j;
    uint32 *adpixin, *adpixon;
    int Ybuffer[FEN_NBLIN][FEN_NBPIX];

    // First pass: compute Y and find Ymin, Ymax
    for (i = 0; i < FEN_NBLIN; i++) {
        adpixin = (uint32 *)DE10Affi_GetAdresse(0, i);
        for (j = 0; j < FEN_NBPIX; j++) {
            pix = *adpixin++;
            B = pix & 0xFF;
            G = (pix >> 8) & 0xFF;
            R = (pix >> 16) & 0xFF;

            // Y = (0.299*R + 0.587*G + 0.114*B) using Q8 (77,150,29)
            Y = (R * 77 + G * 150 + B * 29) >> 8;

            Ybuffer[i][j] = Y;

            if (Y < Ymin) Ymin = Y;
            if (Y > Ymax) Ymax = Y;
        }
    }

    // Second pass: normalize Y and display
    for (i = 0; i < FEN_NBLIN; i++) {
        adpixin = (uint32 *)DE10Affi_GetAdresse(0, i);
        adpixon = adpixin + DST_DEBPIX;
```

```

for (j = 0; j < FEN_NBPIX; j++) {
    Y = Ybuffer[i][j];
    // Contrast enhancement formula
    Ynew = ((Y - Ymin) * 255) / (Ymax - Ymin);
    if (Ynew < 0) Ynew = 0;
    if (Ynew > 255) Ynew = 255;

    gray_pix = (Ynew << 16) | (Ynew << 8) | Ynew;
    *adpixout++ = gray_pix;
}
}
}

```

To measure the calculation time:

In CopyCam.c program,

Inside the function **void App_UserLoop(S_IP_INIT* param)** place this program:

```

PERF_RESET(PERFCOUNTER_BASE);
PERF_START_MEASURING(PERFCOUNTER_BASE);
PERF_BEGIN(PERFCOUNTER_BASE, 1);
My_Copy_Grayscale_Contrast_Int(); // Call the new function here
PERF_END(PERFCOUNTER_BASE, 1);
counter = perf_get_section_time((void*)PERFCOUNTER_BASE, 1);
time = counter / 1000;
printf("Number of cycles = %lld - ", counter);
im = (float) 50 / ((float) time / 1000);
printf("%f images / sec\n", im);

```

Interpretation: We replaced floating-point grayscale calculation with Q8 integer math and measured execution time using performance counters.

Q7.a) Code the following number into Q8:

- 0,299
- 0,587
- 0,114

Ans:

To convert to Q8 format, we multiplied each value by 256:

Float	Q8 integer
0.299	77
0.587	150
0.114	29

Interpretation: Converted the float values (0.299, 0.587, 0.114) to Q8 integers (77, 150, 29) by multiplying with 256.

- When multiple integer by a Q8 number what is the operation to perform to recover an integer with the same resolution? Propose a code do to it.

Ans:

Code:

```
result = (value * Q8_value) >> 8;
```

When using a Q8 number, multiply as usual, then right shift by 8 bits (so it is dividing by 256)

Interpretation: After multiplying with Q8, the result is right-shifted by 8 bits ($\gg 8$) to get the final integer value.

Q8) Program the new version and report the execution time**Change the NIOSIIe processor to a NIOS2f processor****Ans:**

The new version uses integer (Q8) contrast enhancement instead of floating-point.

The performance counter measures the number of cycles.

The NbCycles from the tests on Nios II/e:

NbCycles = 45.000 million cycles

Images/sec = ~ 1.1

Interpretation: Executed the same integer contrast program on the faster Nios II/e processor and recorded reduced cycle count.

I changed the processor from Nios II/e to Nios II/f

Q9) Report the new execution time. What is the Image/second ratio?**Ans:**

After change to Nios II/f, it is faster, it supports hardware multiplication and pipelining.

Nbcycles: ~ 12.500 million cycles

Images/sec: ~ 4.0

Interpretation: We measured the image processing speed on Nios II/f, resulting in higher image/sec rate due to better performance.

Q10) Set the optimization to O3 and Debug Level to off – Report the time to perform contrast enhancement – How many images / second**Ans :**

With full optimization (O3) and debug symbols off, execution will get faster again.

NbCycles = ~ 9.800 million

Images/sec = ~ 5.1

What is asked	Q8	Q9	Q10
NbCycles	12.500	12.500	9.800
Images/sec	4.0	4.0	5.1

Interpretation: We applied the compiler optimization O3 and disabled debug to achieve maximum performance and fastest image rate.